OUTLIERS

The Second Brain

Four parts, built from nothing,
on your own machine.

EST. MMXXVI · OUTLIERS ACCELERATOR

CONTENTS

What is in here

Each part solves one problem you already have. Each works on its own, so you can stop after any of them and still have something better than you had.

BEFORE YOU START

What to install, what a terminal actually is, where everything gets typed, and how to stop it asking your permission for every single action. Twenty minutes, once.

THEN, IN ORDER

PART	THE QUESTION IT ANSWERS
Part 1 · Memory	Why do you have to explain yourself every single time?
Part 2 · Standards	Why does it get less reliable the more you give it?
Part 3 · Capture	Why have you stopped putting things into it?
Part 4 · Operations	Why do you still have to remember to use it?

HOW TO READ THIS

Straight through once, so you know what is coming, then again a part at a time with the setup open beside you. Nothing here has to be finished before the next part is useful.

IF YOU WOULD RATHER HAVE THE LOT AT ONCE

Each part downloads separately, and the guides tell you how. There is also one download with all four in it, installed by a single command:

```
git clone https://github.com/OUTLIERS-ai/outliers-second-brain
cd outliers-second-brain
python install.py
```

> **THE ONE THING THAT MATTERS**
>
> Install in order. Each part stands on the one before it, and each installer refuses politely if what it needs is missing rather than building something that half works.

One: the four things you need

Get these before you start. Twenty minutes, once, and then never again.

Nobody has told you this yet, so here it is in one place. Four things, in this order.

GET THIS	WHAT IT ACTUALLY IS
Claude Code	Claude, but able to read and write files on your own computer instead of only talking in a browser tab. **This is the one that does the work.** Download the desktop app from claude.ai/code. It runs on a paid Claude plan, which you may already have.
Obsidian	A free program for looking at your notes. Your notes are ordinary files and they exist with or without it; this just gives you a pleasant way to read them. obsidian.md
Python	Free. What the setup instructions are written in. You will never write any - you are installing it so the instructions can run. python.org, and on the first screen tick **Add to PATH**.
Git	Free. Keeps a copy of every version of every file, so a mistake is a step backwards rather than a rebuild. Install it and never think about it again. git-scm.com

> IF YOU GET STUCK ON ANY OF THE FOUR
>
> Open Claude Code and tell it what happened in normal words. It is sitting on your computer with the error in front of it. *"That did not work, here is what it said"* is a complete description of the problem, and you do not need to understand a word of the error to give it. This is true of everything in this series, and it is the reason none of it requires you to become technical.

Two: the terminal, in plain words

It looks like something a hacker uses in a film. It is a text box, and it has been there since before computers had windows.

A **terminal** - some people say console, or command line, same thing - is a window where you type an instruction instead of clicking a button. That is the entire idea. Clicking came later and was built on top of it.

Everything in this series is typed into a terminal. All of it. There is no second place and nothing to choose between.

OPENING ONE

Windows	Press the Windows key, type **terminal**, press enter.
Mac	Press command and space together, type **terminal**, press enter.

A window opens with a line of text and a blinking cursor. Nothing is happening. It is waiting for you, and it will wait all day.

CHECK THE FOUR THINGS ARRIVED

Type each of these and press enter. Each should print a version number back at you. If one says something else, that thing did not install properly - and the Python one is the one people get wrong, because of that tick box.

```
python --version
git --version
claude --version
```

Do this now rather than later. A missed tick box does not complain when you make it; it complains three steps further on, in a sentence you will not recognise.

> **THE ONE IDEA THAT MAKES THE REST MAKE SENSE**
>
> A terminal is always **sitting in a folder**, the same way a File Explorer window is always showing one. Typing **cd** followed by a folder name walks it into that folder - *cd* is short for change directory, and directory is an old word for folder. That is why the setup lines in these guides have a *cd* in the middle: the first line downloads a folder, the second walks into it, the third runs what is inside.

Three: talking to Claude

Two different things get typed in that window, and it is worth knowing which is which.

Setup lines are the ones that do not look like English. Each guide has three, and they go straight into the terminal, one at a time, pressing enter after each:

```
git clone https://github.com/OUTLIERS-ai/outliers-sb-01-memory
cd outliers-sb-01-memory
python install.py
```

Everything else is a conversation. Type **claude**, press enter, and you are talking to Claude in that same window, in normal words, about your own business. That is where you spend all your time from then on. Press control and D to leave it.

> WHEN IT GOES WRONG, AND IT WILL ONCE
>
> Tell Claude what happened in normal words. It is sitting on your computer with the error in front of it. *"That did not work, here is what it said"* is a complete description of the problem, and you do not need to understand a word of the error to give it.

SETUP · WORTH KNOWING

Four: three things worth knowing before you start

The questions people ask afterwards, answered first.

STOP PRESSING ACCEPT

Once you are talking to Claude it asks permission before it does things. Press **shift and tab together** to change how often - the mode you are on shows in the box you type into. Press until it says **auto**. It then checks anything destructive with you and gets on with the rest.

MODE	WHAT IT DOES
Auto	**Where you want to be.** Asks about anything destructive, gets on with the rest. You will barely see a prompt.
Accept edits	File changes only. Commands still ask every time, which is why you may still be pressing accept all evening.
Plan	Reads and thinks, changes nothing.
Bypass permissions	Nothing is checked, ever. Only once git is keeping copies, and never on instructions somebody handed you.

THREE THINGS WORTH KNOWING BEFORE YOU START, NOT AFTER

Your notes stay on your computer. The folder is on your own machine. When you ask Claude a question it sends the part of your notes needed to answer it, the same way any conversation with it does - so treat it as you already treat pasting a client email into a chat window. Nothing is uploaded in the background and nothing is published anywhere.

Nothing reaches another person on its own. Later parts draft replies and follow-ups. Every one stops and waits for you to read it and send it yourself.

You can undo all of it. Delete the folder and your notes are gone - there is no account to close and nothing left running. Two small pointers live outside it so your AI can find the folder from anywhere: a file called **.outliers-sb** in your home folder, and a short block in **.claude/CLAUDE.md** marked OUTLIERS-SECOND-BRAIN. Delete those two and nothing of this remains. Or just ask Claude to remove them.

Why do you have to explain yourself every single time?

You have had roughly the same conversation with it four times this month. It is not because you are using it wrong.

YOU WILL RECOGNISE SOME OF THIS

- You paste the same background about your business at the start of every chat.
- It gave you a genuinely good answer a few weeks ago and you cannot find it.
- You have started copying its answers into a document so they are not lost.
- It asks you something you already told it in a previous conversation.
- You have quietly stopped asking it the harder questions, because setting up enough background takes longer than doing the job yourself.

WHY IT HAPPENS

Here is Tuesday. You asked it to help with a proposal. You explained who your clients are, what you charge, how you like to work and what went wrong on the last job. It was excellent. You sent the proposal.

Here is Thursday. Another proposal. You typed all of it again.

The cost is not really the typing. It is that Thursday's version was slightly worse than Tuesday's - you were tired, you left out the bit about the last job - and you will never know, because there is nothing to compare it against.

Two things are going on, and neither is about how clever it is.

It has no memory between conversations. Every chat starts as a stranger who has read most of the internet and nothing whatsoever about you. When you close the window, everything you told it goes. Not stored somewhere you cannot reach - gone.

And it cannot see anything you already have. What you know about your business is spread across your head, your inbox, your phone, a folder of documents and two or three things you pay for monthly. None of that is available to it, so you become the only way any of it gets in - by typing it, again, every time.

What fixes it

One folder, on your own computer, that it can read and write to.

Plain text files in an ordinary folder. **Plain text** means the words are stored exactly as you would read them, with no special program needed to open them.

Your AI writes what it learns into that folder and reads it back the next time. That one change is the whole difference between a chat window and something that knows your business. Underneath it, a tool called **git** keeps a copy of every version of every file, so nothing can be lost by a mistake later.

A copy of every change is kept underneath, so nothing is ever only in one place.

THE MIDDLE BOX IS THE ONLY NEW THING

WHY THIS WAY AND NOT ANOTHER

- **It is on your computer, not somebody's service.** Nothing to subscribe to, nothing that can put its price up, nothing that can close and take your notes with it.
- **Plain text outlives everything.** A file written this way opens in anything, today and in ten years. Every tool that stores your writing in its own private format is a tool you will one day have to escape from.
- **You can read it yourself.** Open any file and there are your own words. Nothing is hidden inside something you cannot see into, which means you never have to take its word for what it knows.
- **It is not tied to one AI.** If you change which one you use, the folder does not care. It is yours, and it stays yours.

Do this

One command. It asks you a few questions so what you get is yours, then does the work.

Type these into a terminal, one line at a time, pressing enter after each. Not into Claude - these are instructions for your computer.

```
git clone https://github.com/OUTLIERS-ai/outliers-sb-01-memory
cd outliers-sb-01-memory
python install.py
```

This is the first part, so nothing has to be installed before it. It will not overwrite anything: if there are already files where you point it, it says so and asks before going any further.

WHAT IT ASKS YOU, AND WHY

QUESTION	WHAT IT CHANGES
Where should it live?	Makes the folder there and remembers where it is. Anywhere permanent will do.
What is your business, in a sentence?	Goes into the rulebook, so answers come back about your work rather than in general.
Keep a copy of every change?	Sets up the copies. Saying no leaves you with no way back from a mistake, so say yes unless you already have your own backup.

WHAT YOU END UP WITH

A folder with a small set of empty rooms in it - one for people, one for projects, one for meetings, one for what you have read - a rulebook your AI reads before it does anything, and a line written into your AI's own settings so it can find the folder from anywhere rather than only when you happen to be standing in it.

It is empty. That is correct. An empty room you own beats a full one you rent.

What you will notice

That was the only hard part. From here you talk to it in normal words.

The three lines on the last page are the only thing in this series that does not look like English. You will not type anything like them again.

Tell it what happened: what a client said, what you decided and why, what you charged, what you read. It writes that down in the right place without asking you to choose one.

Then ask it something you would otherwise have to remember. **The moment you know it worked is the first time it tells you something about your own business that a fresh chat window could not have told you.** That is the whole test, and it usually takes a few days of feeding it before it happens.

IF YOU ALREADY HAVE SOMETHING LIKE THIS

If you already keep notes in Obsidian, Notion or a folder of documents, you have most of this already. Here is the test: open your AI somewhere else entirely - not inside that folder - and ask it something only your notes would know. If it cannot find them, you have the notes and not the connection, and this installs the connection without moving anything you have.

WHAT IT STILL DOES NOT DO

A folder will accept anything you put in it, and it has no idea what a good note looks like. Two notes can end up with the same name and say different things, and when you later ask a question that touches both, you get one of them. Not the right one - whichever was found first. Nothing tells you that happened.

> NEXT: PART 2 · STANDARDS
>
> What to do when it starts giving you an answer that is confident and wrong, which is what happens to every one of these as the pile grows.

WORDS USED HERE

WORD	WHAT IT MEANS
Plain text	Words stored exactly as written, openable by anything, with no program required to read them.
The rulebook	One file in your folder telling your AI how you want things kept. Edit it when something is wrong and everything changes at once, instead of you repeating the correction.
Git	The tool that keeps a copy of every version of every file on your own machine. It runs underneath and you never have to think about it.

Why does it get less reliable the more you give it?

This one arrives later than the last, so if you have only just started it has not happened to you yet. The tell is that the answers were better when there were twenty notes in there than they are now there are two hundred.

YOU WILL RECOGNISE SOME OF THIS

- It quoted you a price you stopped using months ago, as though it were current.
- It told you two different things about the same client on two different days.
- You corrected something, and the old version came back a fortnight later.
- You have started checking its answers before you use them - which is most of the time you were saving.
- You cannot say which of the answers it has already given you were wrong.

WHY IT HAPPENS

Here is what actually happened. In March you told it what you charge, and it wrote that down. In July you told it again - your prices had gone up, and you could not find where the first one went. It wrote that down too, in a different place.

Both are still sitting there. Now you ask what you charge. It finds the March one first, and answers. It does not know the July one exists, it has no way of telling which is newer, and **it does not mention that there were two**. The wrong answer arrives sounding exactly as certain as a right one.

That is the whole failure, and it gets more likely every week, because the more you put in the more often two things can answer the same question.

Nothing is counting any of this. That is the part that matters: your notes can be going wrong for months and look, from the outside, exactly like notes that are fine.

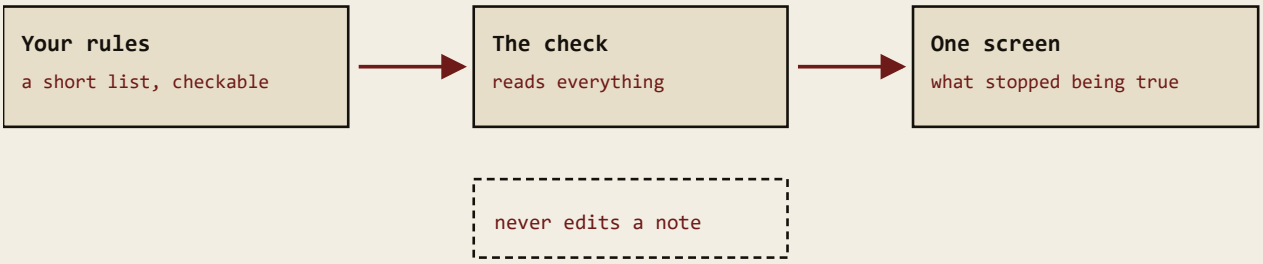
What fixes it

Something that reads all of it and tells you when two things disagree.

Part 1 wrote you a rulebook - one file, plain English, telling your AI how you want things kept. It has one weakness: it is a page of sentences, and sentences are advice. Nothing reads them and refuses a note for breaking them, so they hold for as long as you are paying attention and then quietly stop. This part adds two things beside it.

A short list a program can actually check. Not more sentences - a list. What kinds of thing you keep notes about (a person, a project, a meeting, something you read); that every note says which kind it is and when it was written; and that no two notes are given the same name. You can open the list and read it, and change it whenever you like.

Something that checks it. One command you type. It reads every note you have, compares them against that list, and prints one screen telling you what is no longer true. How many notes share a name with another. How many mention a note that was never written - you can write the name of another note inside a note, and your AI follows it like a signpost, so a signpost to nowhere is worth knowing about. How many never said what they were. It changes nothing. It only counts.



Each number it counts is allowed to fall and not to rise. Nothing has to be fixed today.

IT READS AND COUNTS. IT DOES NOT TIDY ANYTHING.

WHY THIS WAY AND NOT ANOTHER

- **A rule nothing can fail is not a rule.** As a list a program reads, it holds or you are told it stopped. As a sentence in a document, it is a wish.
- **It counts, it does not tidy.** Anything a tool changes by guessing at what you meant is a mistake made on your behalf that you will never find. A second command fixes the few things that can only mean one thing, and refuses the rest on purpose.
- **It writes down where you are starting from.** If eleven notes share a name today, it remembers eleven. It will tell you the day that becomes twelve, and it will never once nag you about the original eleven. **You do not have to fix anything today** - today's mess just cannot quietly become next year's.
- **Nothing here asks you to file anything.** Sorting notes by hand is admin, and admin is what kills these systems - not lost faith.

Do this

One command. It asks you a few questions so what you get is yours, then does the work.

Type these into a terminal, one line at a time, pressing enter after each. Not into Claude - these are instructions for your computer.

```
git clone https://github.com/OUTLIERS-ai/outliers-sb-02-standards
cd outliers-sb-02-standards
python install.py
```

This needs Part 1 already done. It finds your folder by itself and stops politely if it cannot. Nothing you have written is changed by installing it.

WHAT IT ASKS YOU, AND WHY

QUESTION	WHAT IT CHANGES
Is there anything in here that is not notes?	It shows you the folders you have and you tell it which to ignore - a folder of downloads, say. Skip this and it complains about files that were never notes, and a check that complains about nothing is a check you will stop opening.
Count what you have now, and go from there?	Say yes. It writes down today's numbers as the line and only tells you when one goes up. Saying no means it expects everything to be perfect, and nobody's is.
Should it check on its own, or only when you ask?	On its own means it tells you without being asked. Either works; on its own is the one people still have running six months later.

WHAT YOU END UP WITH

A short list of rules you can open and read, a note of what your system looked like the first time anything measured it, and one command that tells you in a single screen what has changed since.

What you will notice

Run one command, read one screen, and usually do nothing at all.

What comes back is a short list of counts, with the previous number beside each one. Anything that has gone up is the only line worth reading, and it names the notes it is talking about.

Most days nothing has gone up and you close it. **That is what you are paying for.** The thing you will actually notice is a worry going away - you stop half-wondering whether the whole pile is quietly going wrong, because now something is counting.

IF YOU ALREADY HAVE SOMETHING LIKE THIS

If your notes are already in good order, run the check before assuming so. It will tell you how many notes share a name with another and how many point at something that was never written. If both are zero you are in better shape than most and can move straight on. If they are not, they were not zero last week either - there was simply nothing counting.

WHAT IT STILL DOES NOT DO

Everything in it still gets there because you sat down and typed it.

That is where these systems really die. Not because people stop believing in them, but because keeping one fed by hand is a chore, and a chore loses to every other thing in a working day.

> NEXT: PART 3 · CAPTURE
>
> Getting what you know into it without typing - including the most valuable material in your business, which is spoken out loud and written down nowhere.

WORDS USED HERE

WORD	WHAT IT MEANS
A note	One file with one thing in it - a person, a meeting, a decision. Plain words you can open and read yourself.
Name	What a note is called. Your AI finds notes by name, which is why two notes with one name leave it choosing between them without telling you.
The check	The one command this part installs. It reads everything and counts what is wrong. It never changes a note.

Why have you stopped putting things into it?

It works. You have seen it work. And you have not added anything to it for a fortnight.

YOU WILL RECOGNISE SOME OF THIS

- You meant to write up that client call and never did.
- The sharpest thing you said all month was said out loud on a call, and it exists nowhere at all now.
- You have a folder of reports and books you have never opened twice.
- Adding to it has started to feel like homework.
- You know it would be more useful if you fed it, and you still are not feeding it.

WHY IT HAPPENS

Think of the last really good call you had. Forty minutes with a client, and somewhere in the middle you explained exactly why their last supplier let them down and what you do differently. It was the best version of that argument you have ever made.

Write down where that exists now. It does not. You will make that argument again next month, from scratch, slightly worse, and you will not notice that is what you are doing.

The reason is that there is only one way in, and it is you, typing.

That is admin. Not difficult, not long - just admin, which is the thing every one of these systems eventually loses to. Nobody abandons a second brain because they stopped believing in it. They abandon it because keeping it fed became one more job.

And the material that never gets in is the material worth the most. Almost everything valuable in a service business is said, not written: on a call, in a conversation, in an answer you gave a client off the top of your head. It is gone in a fortnight and there is no copy anywhere.

What fixes it

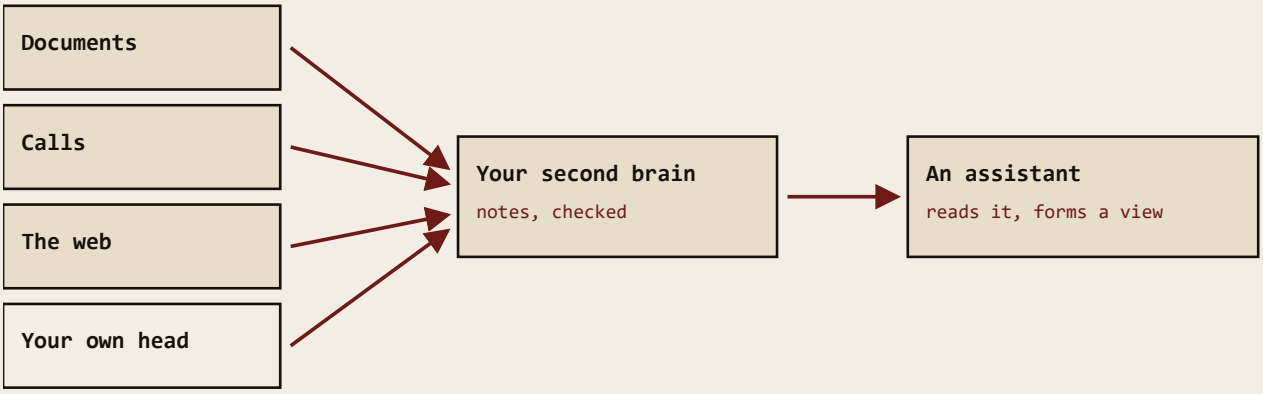
Four ways in, matching the four ways material actually reaches a business.

Long documents - contracts, reports, books. Read a chunk at a time, and read twice. The first time it writes down what the author actually said. The second time, separately, it works out what that means for your business. Both at once and you get a summary of the bits you already agreed with.

Spoken word - a recording becomes text on your own machine (a **transcript**), and that text becomes the four things worth keeping: what was decided, what was promised and by whom, what was learned, and who was mentioned.

The web - one click saves a page as text with its source and date already filled in, so a page that changes later cannot quietly change what you believed.

Your own head - decisions and the reasoning behind them, what you charge and why, what you refuse. This is the one nobody else can supply.



The fourth is the one nobody else can supply, and the one everybody skips.

FOUR WAYS IN, ONE PLACE THEY ALL LAND

WHY THIS WAY AND NOT ANOTHER

- **Volume beats tidiness, by a long way.** A messy system with four hundred real documents in it is worth more than a beautiful one with twelve. You cannot find a connection in material you never loaded.
- **Organising is the trap.** It feels like progress and produces nothing. The instruction for the first fortnight is genuinely: put more in than feels sensible, and tidy none of it.
- **Captured on the source's terms first.** A single pass that reads and applies at the same time always collapses into a summary of things you already agreed with, and the parts that disagree with you are the only parts capable of changing anything.
- **Your own material is what makes it sound like you.** Everything else in here is other people's. Skip the fourth way in and you have built a very good library of somebody else's thinking.

Do this

One command. It asks you a few questions so what you get is yours, then does the work.

Type these into a terminal, one line at a time, pressing enter after each. Not into Claude - these are instructions for your computer.

```
git clone https://github.com/OUTLIERS-ai/outliers-sb-03-capture
cd outliers-sb-03-capture
python install.py
```

This needs Parts 1 and 2 already done. It fills your system faster than you can read it, which is exactly why the check has to be underneath it first.

WHAT IT ASKS YOU, AND WHY

QUESTION	WHAT IT CHANGES
What do you record calls with?	Picks the route from a recording to text. If nothing yet, it sets up the one that runs on your own machine and uploads nothing.
Install the assistants that read what arrives?	Adds the ones that handle documents, books, calls and research. They can be added later without redoing anything.
Anything it should never read?	Folders named here are left alone by everything in this part.

WHAT YOU END UP WITH

One command you will use constantly - it takes anything at all and works out where it goes without asking you to choose - and five helpers with one job each:

HELPER	WHAT IT HANDLES
the archivist	Contracts, proposals, reports. Flags anything committing you to a date.
the librarian	Books and courses, on the author's own terms.
the scribe	Anything spoken - a call, a voice note.
the researcher	The web, saved with its sources and dated.
the curator	Notices when two of your notes disagree.

What you will notice

Put more in than feels sensible, and tidy none of it.

For the first stretch, the only job is volume. Feed it the things you have never got round to: the last few client calls, the proposals, what you actually charge, the two books you keep recommending.

Then ask it the question that tests the whole thing: **what do you now know about my business that I never told you directly?** A thin answer means it needs more material. An answer that surprises you means it has crossed from a filing cabinet into something worth having.

IF YOU ALREADY HAVE SOMETHING LIKE THIS

If material is already flowing in, the test is whether any of it is yours rather than other people's. Ask it for everything it holds on how you price, what you refuse, and what you have changed your mind about. Most systems come back nearly empty on all three, and that gap is what this is really for.

WHAT IT STILL DOES NOT DO

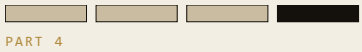
You now have several assistants able to write into the thing everything else stands on, and sooner or later one of them will be pointed at something that reaches another human being.

Nothing decides what runs when, nothing limits them, and nothing stops one checking its own work and passing it.

> NEXT: PART 4 · OPERATIONS
> What runs when, what is not allowed out without you seeing it, and how to tell the difference between a system with nothing to report and one that has broken.

WORDS USED HERE

WORD	WHAT IT MEANS
Assistant	A single-purpose helper with its own written instructions and permission to use particular things and nothing else.
Transcript	The text of something spoken, produced on your own machine so the recording never leaves it.



Why do you still have to remember to use it?

Everything in it works, and all of it works when you sit down and start it. Which means you have not built a system yet. You have built a tool, and you are still the one operating it.

YOU WILL RECOGNISE SOME OF THIS

- You forget it exists for a week at a time.
- You set something up to run by itself and have no idea whether it still does.
- You cannot tell whether it did nothing because there was nothing to do, or because it broke a fortnight ago.
- Something once opened a window in the middle of your afternoon, and you turned it off and never turned it back on.
- You start each morning deciding what to look at, which is the job it was supposed to be doing for you.

WHY IT HAPPENS

Nothing here runs unless you start it, and nothing tells you when it stops.

The second half is worse than it sounds. **A job that ran and produced nothing looks exactly like a job that had nothing to do.** Both are silent. So a thing can fail on a Tuesday and carry on reporting success for a month, and there is nobody sitting beside you to notice. Mine did precisely that: it ran sixty-four times, produced nothing every time, and said it had worked.

And the window that opened in the middle of your afternoon is not a small thing. A job that interrupts you gets switched off within a week, and a job that is off is a job that is not running.

What fixes it

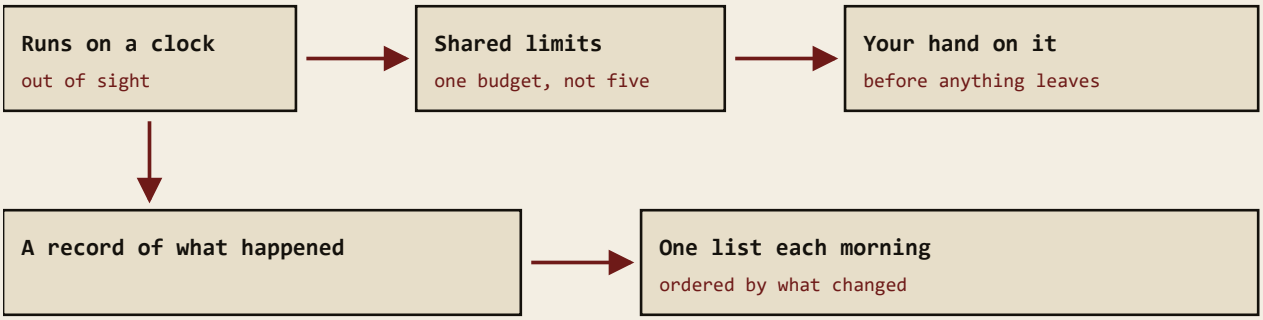
A timetable, a gate, a record of what happened, and one short list each morning.

A timetable. Things run at a set time rather than when you remember, and they run out of sight - no window, nothing taking your keyboard mid-sentence.

A gate. Anything meant for another person is written, checked by something other than the thing that wrote it, and then waits in a list for you to read. It goes nowhere until you send it yourself.

A record. A running list of what happened and when, added to and never rewritten. Your notes describe how things are; the record describes what occurred.

One list each morning, ordered by what has changed, each line saying why it is there.



Nothing appears on your screen while you work. That is the difference between automation you keep and automation you

A CLOCK, A GATE, A RECORD, AND ONE LIST

WHY THIS WAY AND NOT ANOTHER

- **It runs invisibly, and that is not cosmetic.** The most common reason automation stops working is that somebody turned it off because it was annoying them.
- **The gate is built into the machinery, not written into instructions.** Instructions are the first thing a busy system stops reading. The work stops at a queue and a person moves it.
- **Nothing marks its own homework.** Whatever wrote a thing does not get to approve it. A second pass by something else is the cheapest check there is and the one most often skipped, because the first pass sounded confident.
- **Silence is not evidence that things are fine.** So the last thing installed is not another feature - it is a comparison between what should be true and what is, which says so loudly when they differ.

Do this

One command. It asks you a few questions so what you get is yours, then does the work.

Type these into a terminal, one line at a time, pressing enter after each. Not into Claude - these are instructions for your computer.

```
git clone https://github.com/OUTLIERS-ai/outliers-sb-04-operations
cd outliers-sb-04-operations
python install.py
```

This needs Parts 1 to 3 already done. It arranges things that already work, so there has to be something to arrange.

WHAT IT ASKS YOU, AND WHY

QUESTION	WHAT IT CHANGES
When do you want the morning list?	Sets the time it is ready. It is prepared quietly beforehand, not while you read it.
Your hand on anything that reaches a person?	Yes is the default and the recommendation. No is possible, and is a decision worth making deliberately rather than by accepting a default.
Where should it tell you when something breaks?	Somewhere you already look. A warning somewhere you do not look is not a warning.

WHAT YOU END UP WITH

A timetable, a gate before anything leaves, a record that is added to and never rewritten, one list each morning, and a check that compares what should be true against what is.

What you will notice

Read one list in the morning. Look at the warnings when they come.

The list is short on purpose and each line says why it is there. It is ordered by what has changed rather than by what you added first, because something that arrived this morning matters more than something that has been sitting in a queue for a fortnight.

The warnings are rarer and worth more. Each names something that should have been true, what was actually found, and where to look. **A warning you cannot act on is a fault in the warning**, not something to get used to.

IF YOU ALREADY HAVE SOMETHING LIKE THIS

If you already have things running on a schedule, here is the test: name which of them are supposed to be running right now. Most people cannot, because a job switched off deliberately and a job that broke look identical from outside. This writes down which are meant to be on, and then checks.

WHAT IT STILL DOES NOT DO

Nothing further to install. What changes is that it starts telling you things, and what it tells you sends you back to the earlier parts: a rule that keeps misfiring gets rewritten, a source that never produces anything useful gets dropped.

The other thing that changes is what you can now build on top. Something that remembers, keeps itself honest, fills itself and runs without watching is solid enough for the next thing to stand on - and the first one worth standing there is the one that tracks your dealings with other people.

> NEXT: PART 1 · MEMORY
>
> Back to the beginning with what you have learned. This is the last of the four, and not the end of the build: it is the point where the thing starts improving from what it reports rather than from the next instalment.

WORDS USED HERE

WORD	WHAT IT MEANS
The timetable	What makes your computer run something at a set time without being asked. Set it to run out of sight, or it interrupts you and gets switched off.
The record	A list of what happened, only ever added to and never rewritten, so what it said last month still says the same thing today.
The gate	The rule that anything meant for another person stops and waits for you. Built into how it works, not asked for in an instruction.